

Smart Fuel Management System

1. Project Overview

The Smart Fuel Management System is a microservices-based backend system designed to manage fuel stations, fuel sessions, user accounts, transactions, vehicles, payments, notifications, and authentication.

The system is built using distributed system principles to improve:

- Scalability
- Fault isolation
- Maintainability
- Security
- Performance

The architecture uses:

- API Gateway
 - Service Registry
 - Authentication Service
 - Multiple Microservices
 - Redis Caching
 - Message Queues
 - External Payment and Notification Providers
-

2. Objectives

Main goals of the system:

- Manage users and fuel stations
 - Handle fuel sessions and transactions
 - Support secure payments
 - Send email and SMS notifications
 - Improve performance using Redis caching
 - Support scalability using microservices architecture
 - Provide centralized authentication and routing
-

3. System Architecture

Main Components

Client Applications

The system supports multiple clients:

- Web Application
- Mobile Application
- Administrative Dashboard

Clients communicate with the backend through the API Gateway.

API Gateway

Responsibilities:

- Single entry point for all requests
- Request routing
- Authentication forwarding
- Security filtering
- Rate limiting
- Load balancing

Benefits:

- Centralized traffic management
 - Simplified client communication
 - Improved security
-

Authentication Service

Responsibilities:

- User authentication
- JWT token validation
- Permission checking
- Authorization

Security Features:

- BCrypt password hashing
- JWT authentication

- Role-based access control
 - Token validation
-

Service Registry

Responsibilities:

- Service discovery
- Dynamic registration of services
- Tracking active service instances

Possible Technologies:

- Eureka Server
-

4. Microservices

User Service

Responsibilities:

- User management
- User profile updates
- User information retrieval

Main Operations:

- Create user
 - Update profile
 - Get user data
 - Delete user
-

Station Service

Responsibilities:

- Fuel station management
- Station information
- Station status tracking

Main Operations:

- Create station
 - Update station
 - View station data
 - Monitor station activity
-

Wallet Service

Responsibilities:

- Wallet balance management
- Wallet transactions
- User credits

Main Operations:

- Add balance
 - Deduct balance
 - View wallet history
-

Password Reset Token Service

Responsibilities:

- Password reset token generation
- Token expiration validation
- Secure password recovery

Flow:

1. User requests password reset
 2. Token generated
 3. Email sent using messaging queue
 4. User resets password
-

Transaction Service

Responsibilities:

- Store transaction records
- Payment history
- Financial logs

Main Operations:

- Record transactions
 - Retrieve transaction history
 - Validate payment records
-

Vehicle Service

Responsibilities:

- Vehicle registration
- Vehicle ownership tracking
- Vehicle fuel history

Main Operations:

- Add vehicle
 - Update vehicle
 - Retrieve vehicle information
-

Payment Preference Service

Responsibilities:

- Store user payment preferences
 - Manage default payment settings
-

Payment Method Service

Responsibilities:

- Manage linked payment methods
- Store card references securely

Security:

- No sensitive card data stored directly
 - Uses secure payment provider tokens
-

Fuel Session Service

Responsibilities:

- Handle fuel session lifecycle
- Track fueling operations
- Monitor active sessions

Main Operations:

- Start fuel session
 - End fuel session
 - Store session details
-

Payment Service

Responsibilities:

- Process payments
- Communicate with external payment providers
- Validate payment status

External Integration:

- Stripe
-

5. Database Design

Main Database

Stores:

- Users
- Stations
- Transactions
- Vehicles
- Wallets
- Fuel sessions

Possible Technology:

- PostgreSQL
- MySQL

6. Redis Caching

Redis is used to improve system performance.

Responsibilities:

- Cache frequently accessed data
- Reduce database load
- Improve response times
- Store temporary session data
- Support rate limiting

Caching Flow:

1. Service checks Redis cache
2. If cache hit → return cached data
3. If cache miss → fetch from database
4. Store response in Redis

7. Messaging System

The architecture uses message queues with exchanges and routers.

Possible Technologies:

- RabbitMQ
- Kafka

Benefits:

- Asynchronous communication
 - Improved scalability
 - Reduced service coupling
 - Better fault tolerance
-

8. Notification System

Email Notifications

Flow:

1. Service publishes event to exchange
2. Router sends message to queue
3. Java Mail service consumes queue
4. Email sent to user

Use Cases:

- Password reset
 - Transaction confirmation
 - Account notifications
-

SMS Notifications

Flow:

1. Service publishes event
2. Router forwards message to queue
3. Twilio service sends SMS

Use Cases:

- OTP verification
 - Payment confirmation
 - Security alerts
-

9. Payment Integration

The payment service communicates with Stripe through asynchronous messaging.

Payment Flow:

1. User initiates payment
2. Payment service validates request
3. Message sent through exchange and queue
4. Stripe processes payment
5. Transaction status updated

Security:

- HTTPS communication
 - Tokenized payments
 - Secure API keys
-

10. Security Design

Authentication

- JWT Tokens
 - Access token validation
 - Secure login flow
-

Authorization

- Role-based access control
 - Permission validation
 - Protected endpoints
-

Password Security

- BCrypt hashing
 - Password reset tokens
 - Token expiration
-

API Security

- HTTPS
 - Input validation
 - Secure headers
 - Request filtering
 - Rate limiting
-

Infrastructure Security

- Service isolation
- Gateway protection

- Internal service communication control
-

11. System Workflow Example

Fuel Purchase Flow

1. User logs in
 2. JWT token generated
 3. User starts fuel session
 4. Fuel session service validates station
 5. Payment service processes payment
 6. Stripe confirms payment
 7. Transaction stored
 8. Notification event published
 9. Email/SMS confirmation sent
-

12. Advantages of This Architecture

Scalability

Each service can scale independently.

Maintainability

Services are separated by responsibility.

Fault Isolation

Failure in one service does not stop the entire system.

Performance

Redis caching reduces database pressure.

Security

Centralized authentication and secure communication.

13. Possible Future Improvements

- Kubernetes deployment
 - CI/CD pipeline
 - Monitoring with Prometheus and Grafana
 - Distributed tracing
 - API documentation using Swagger/OpenAPI
 - Docker containerization
 - Kafka event streaming
 - Multi-region deployment
 - AI analytics for fuel prediction
-

14. Suggested Technology Stack

Component	Technology
Backend Framework	Spring Boot
API Gateway	Spring Cloud Gateway
Service Registry	Eureka
Authentication	Spring Security + JWT
Database	PostgreSQL
Cache	Redis
Messaging	RabbitMQ
SMS Provider	Twilio
Email Service	Java Mail
Payment Provider	Stripe
Containerization	Docker

15. Conclusion

The Smart Fuel Management System uses a modern distributed microservices architecture designed for scalability, maintainability, security, and performance.

The system combines:

- Microservices
- API Gateway
- Authentication
- Redis caching
- Messaging queues
- External integrations

This architecture supports real-world enterprise backend engineering practices and provides a strong foundation for future scaling and feature expansion.